NeuroMIMO

FEATURE GUIDE

Every feature, with screenshots, demo steps and what to say

Prepared for the faculty demo of the AI-enhanced MIMO-OFDM channel estimation platform.

October 29, 2026

Contents

1. Demo Plan 2
2. Overview Dashboard and 1-Click Demo 3
3. MIMO-OFDM Simulator 5
4. Channel Estimation Lab 7
5. Signal Visualization 9
6. Performance Comparison 11
7. ML Model Training 13
8. Experiment History 15
9. About & Methodology 16
10. Colour Theme Picker 17
11. Features Behind the Scenes 19
12. If Something Goes Wrong 21

1. Demo Plan

NeuroMIMO has **9 features in the web dashboard** and **6 features behind the scenes**. This guide explains each one with a real screenshot, the steps to show it, a sentence to say, and the question a faculty member is most likely to ask. A complete demo takes about 10–12 minutes.

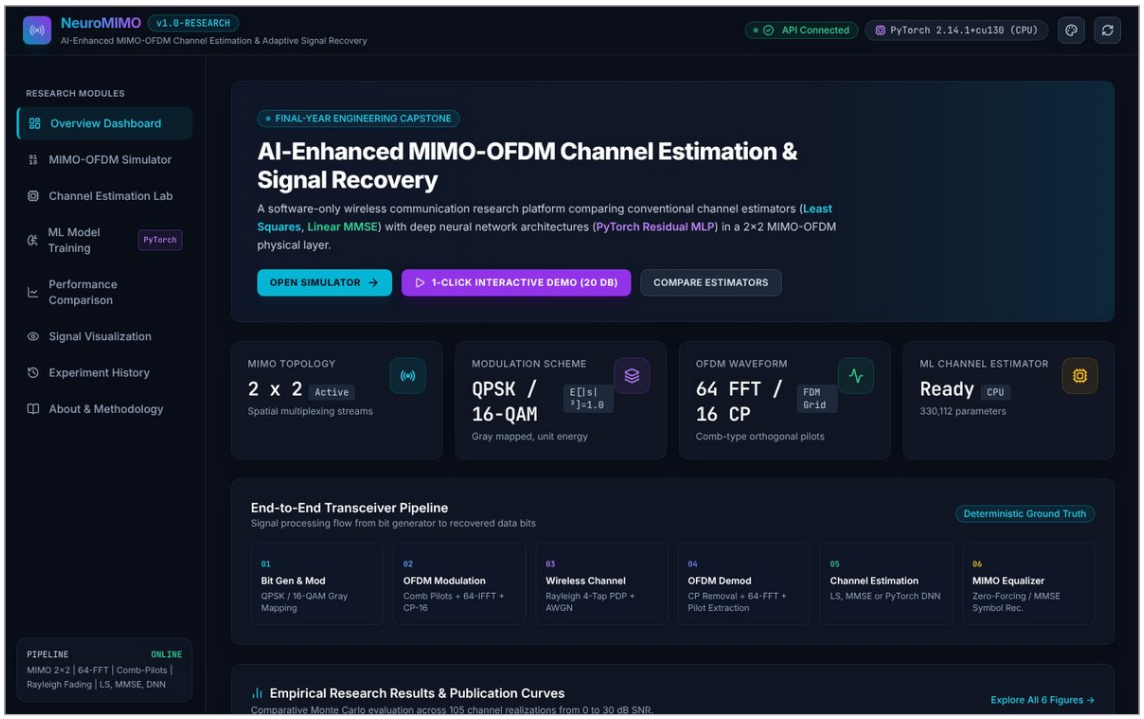
Order	Feature	Time	Main point
1	Overview Dashboard + 1-click demo	1 min	What the project is; a live result in one click
2	MIMO-OFDM Simulator	2 min	Any setting, full transmit–channel–receive chain
3	Channel Estimation Lab	2 min	LS vs LMMSE vs DNN on the same channel
4	Signal Visualization	1 min	Constellation, channel response, pilots
5	Performance Comparison	2 min	The research results over 0–30 dB
6	ML Model Training	1 min	Train the DNN live and watch the loss
7	Experiment History	30 s	Every run is saved and exportable
8	About & Methodology	30 s	Theory and equations
9	Theme picker	15 s	Finishing touch

Before the Faculty Arrives

```
# Terminal 1 (project folder)
uvicorn neuromimo.api.main:app --app-dir backend --port 8000
# Terminal 2
cd frontend
npm run dev                    # open http://localhost:5173
```

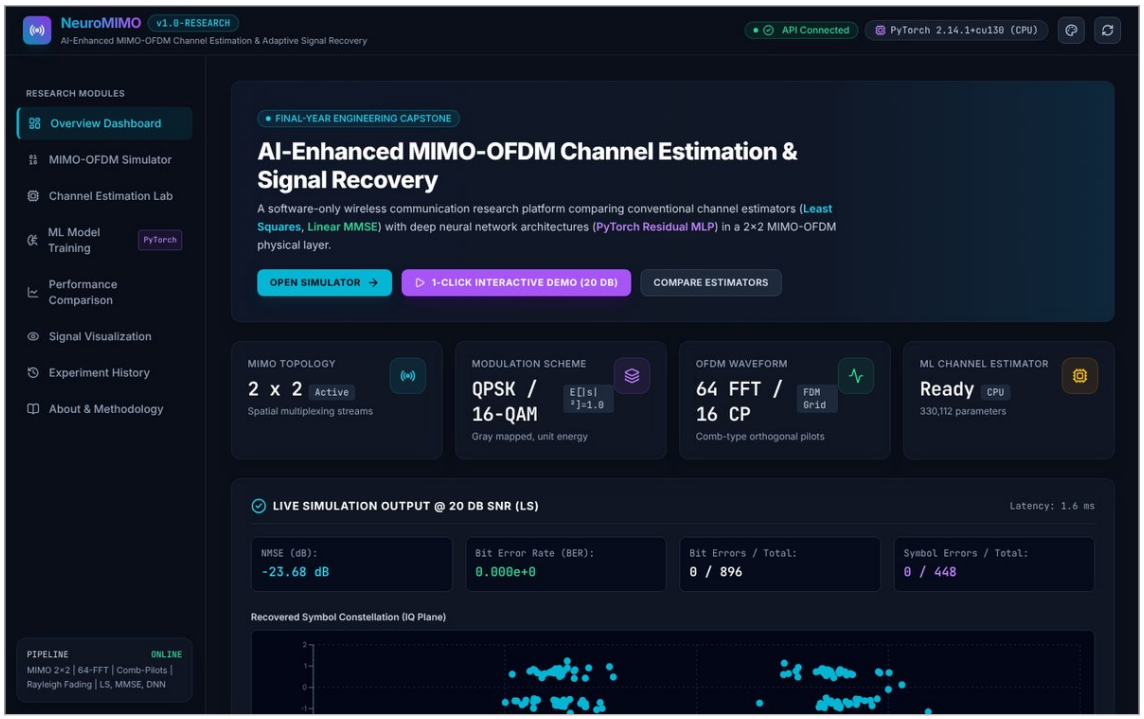
- Check the top bar shows a green **API Connected** badge and the model card says **Ready**.
- **Warm up:** run one simulation and run the Estimation Lab benchmark **twice**. The first PyTorch call after a start or a training run is slow; the second shows the real latency.
- Set the browser zoom so the charts are easy to read from a distance (90–100%).
- Keep the PDFs in `docs/` and this guide's screenshots open as a backup in case the laptop or network fails.

2. Overview Dashboard and 1-Click Demo

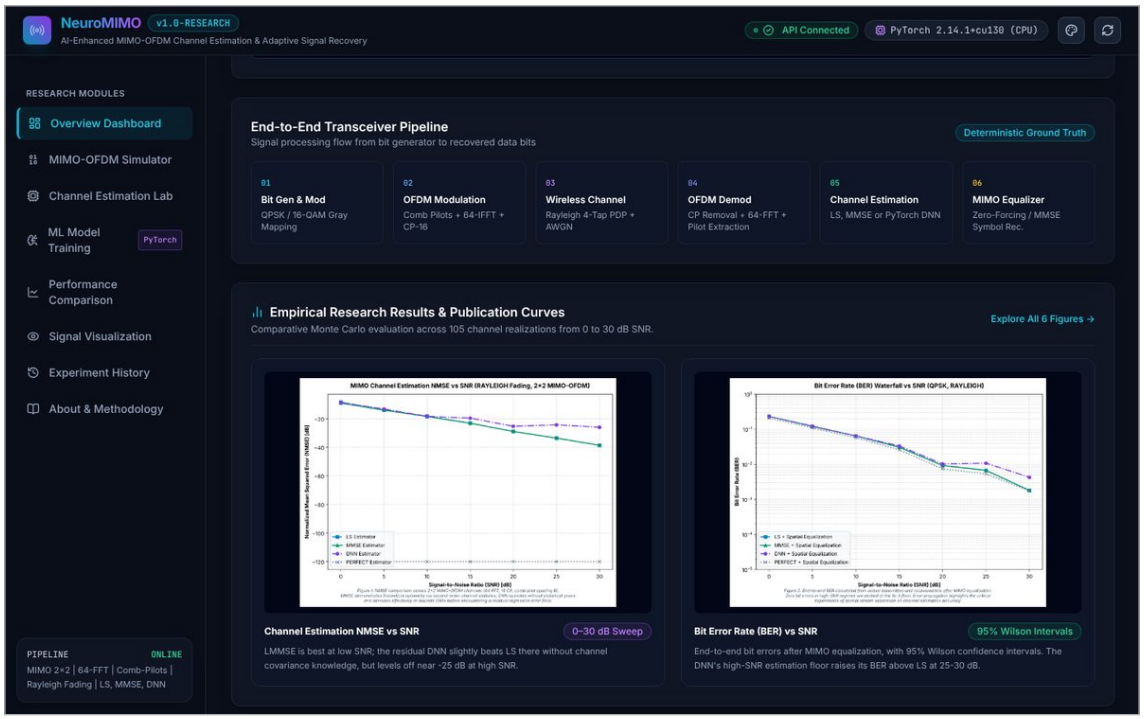


The landing page: project summary, system cards and the transceiver pipeline.

What it is	The home page. It explains the project and shows the system's live status: 2×2 MIMO, QPSK/16-QAM, 64-FFT with 16-sample CP, and whether the trained DNN is ready.
What you see	Four status cards, the six-step transceiver pipeline (bits → OFDM → channel → demodulation → estimation → equalisation), and the main result figures.
How to demo	1. Point to the green API Connected badge and the model card (330,112 parameters). 2. Walk through the six pipeline boxes in one sentence each. 3. Press **1-Click Interactive Demo (20 dB)** and read out the NMSE and bit errors. 4. Scroll down to the NMSE and BER figures.
What to say	*"This platform simulates a complete 2×2 MIMO-OFDM link and compares three channel estimators. One click runs a full transmission at 20 dB and shows the estimation error and the recovered constellation."*
If asked	**Is this real radio data?** No, it is a simulation, which lets us know the true channel exactly. Realistic ray-traced channels can be loaded with the DeepMIMO loader.

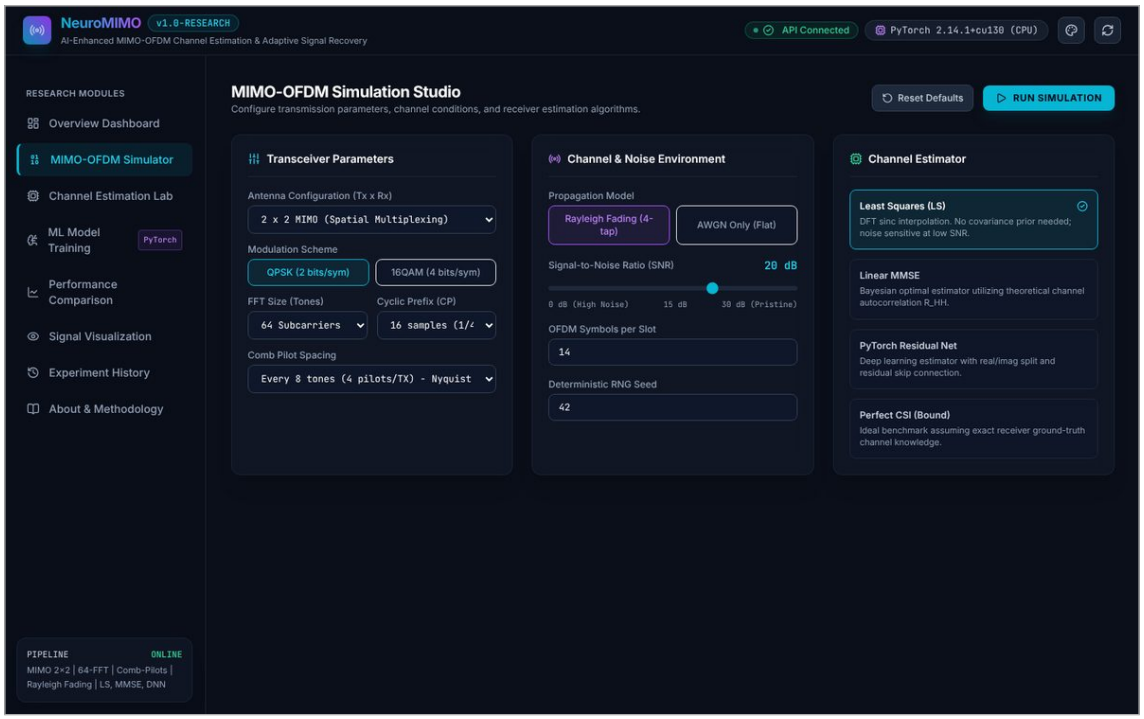


After pressing “1-Click Interactive Demo (20 dB)”: a live result with NMSE, BER, bit errors and the constellation.



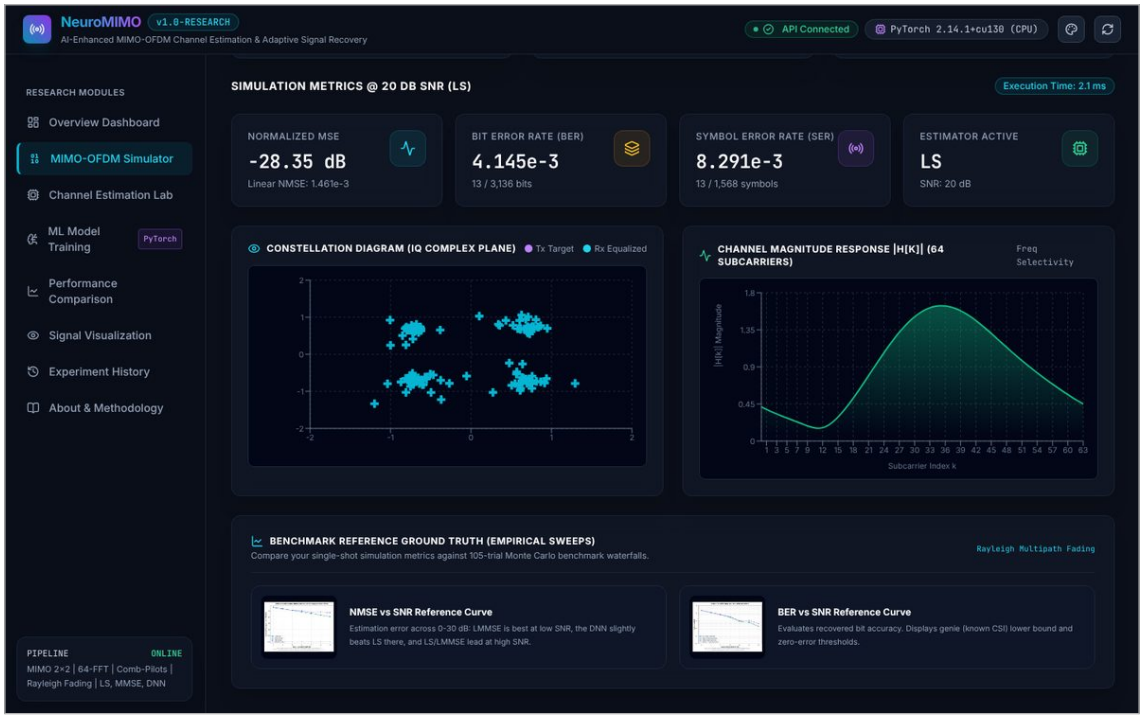
Further down: the research figures with their key takeaway.

3. MIMO-OFDM Simulator



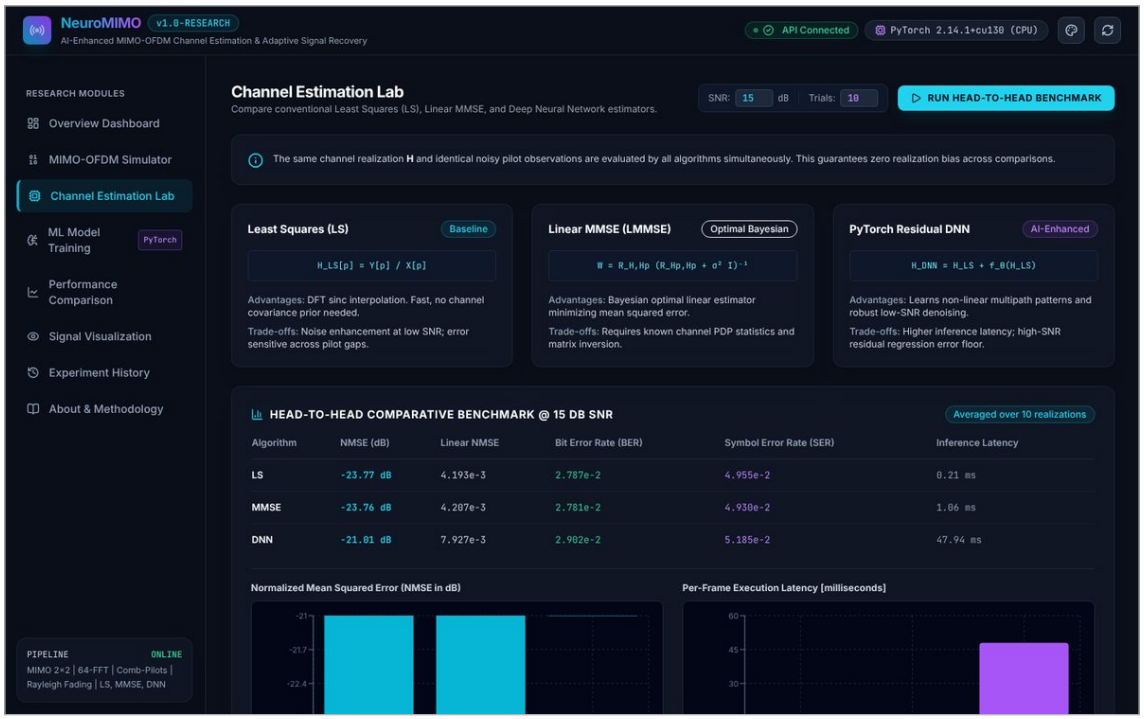
Configuration: transceiver parameters, channel and noise, and the estimator.

What it is	A full single transmission with settings you choose. It runs the 12-step chain: bits, modulation, pilots, IFFT + CP, channel, noise, FFT, pilot extraction, estimation, equalisation, demodulation and metrics.
What you see	Three setting panels and, after running, four metric cards (NMSE, BER, SER, estimator), the IQ constellation and the channel's magnitude across 64 subcarriers.
How to demo	1. Keep 2×2, QPSK, 64 subcarriers, Rayleigh; set SNR to **5 dB** and press **Run Simulation**: the constellation is scattered. 2. Set SNR to **25 dB** and run again: the four QPSK clusters become tight and BER falls. 3. Switch the estimator to **Linear MMSE** or **PyTorch Residual Net** and compare. 4. Point to the channel magnitude plot: it rises and falls across subcarriers.
What to say	*"Each run sends 3,136 bits through a random 4-tap Rayleigh channel. The dips in this curve are frequency-selective fading, which is why the receiver must estimate the channel on every subcarrier."*
If asked	**What is SER?** Symbol error rate: the fraction of QPSK symbols with at least one wrong bit. With 2 bits per symbol it is roughly twice the BER at low error rates.



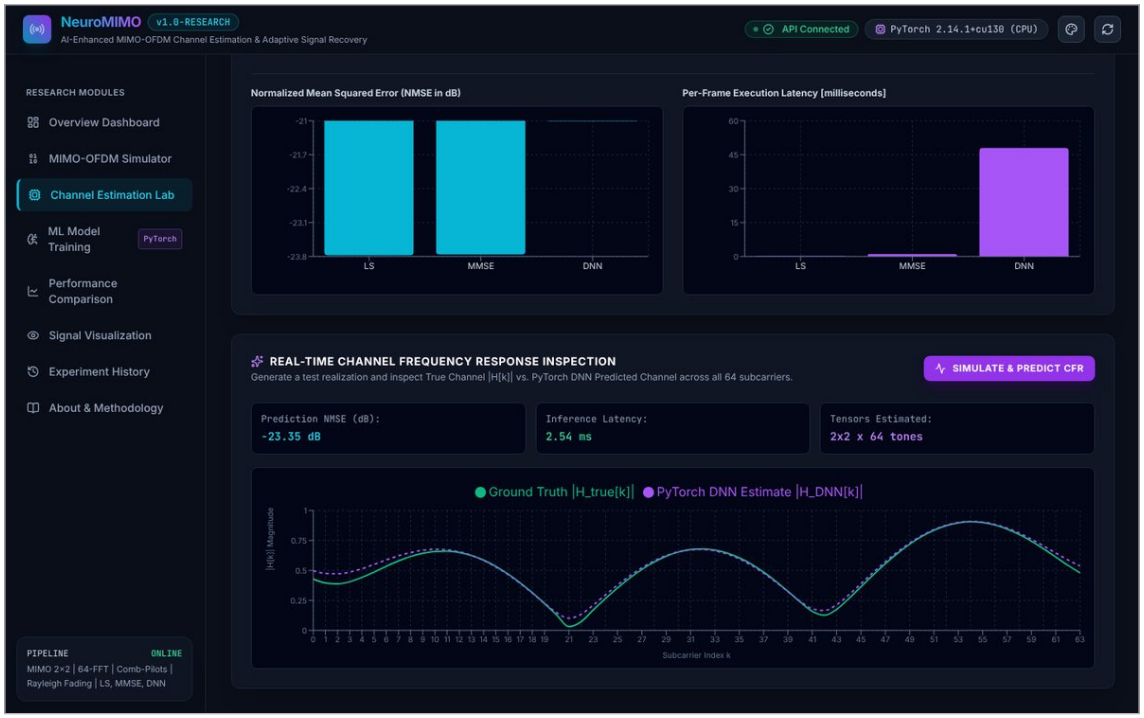
Results of one run at 20 dB with LS: NMSE, BER, SER, constellation and channel magnitude response.

4. Channel Estimation Lab



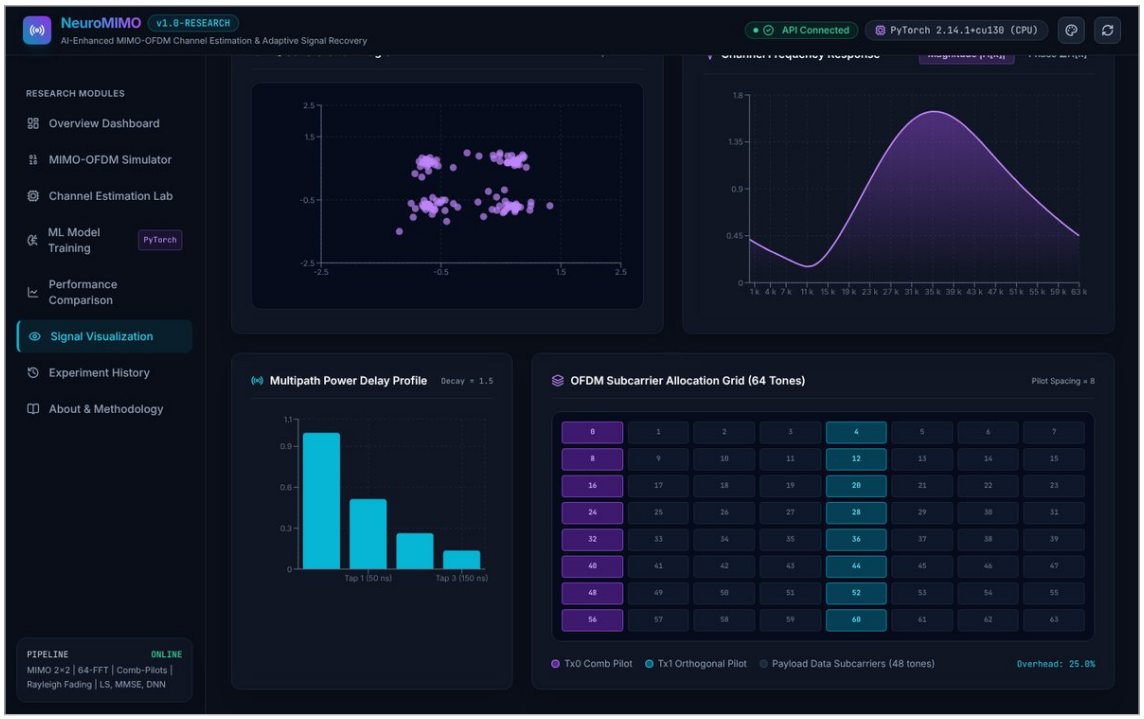
Head-to-head benchmark at 15 dB over 10 channel realisations.

What it is	The core comparison. LS, LMMSE and the DNN estimate **the same** channel from **the same** noisy pilots, so differences come only from the estimators.
What you see	Cards with each estimator's formula, advantages and trade-offs; a results table (NMSE, BER, SER, latency); bar charts of NMSE and latency; and a live plot of the true versus DNN-predicted channel.
How to demo	1. Set SNR to **0** and press **Run Head-to-Head Benchmark**: LMMSE is best (−9.35 dB), then the DNN (−8.96 dB), then LS (−8.50 dB). 2. Set SNR to **25** and run again: LS (−33.28 dB) and LMMSE (−33.25 dB) now beat the DNN (−25.73 dB). The seed is fixed, so you will see exactly these numbers. 3. Press **Simulate & Predict CFR** and show how closely the dashed DNN curve follows the true channel.
What to say	*"At low SNR LMMSE is best because it uses channel statistics; the DNN beats LS without those statistics. At high SNR the DNN levels off, which is its known limitation."*
If asked	**Why is the DNN slower?** It runs a 330k-parameter network. This page shows about 1.5–2 ms per frame because it includes Python and web-request overhead; the dedicated timing test, which measures only the network after warm-up, gives 0.65 ms, inside a 1 ms 5G slot. The first click after a start is slower because PyTorch initialises.



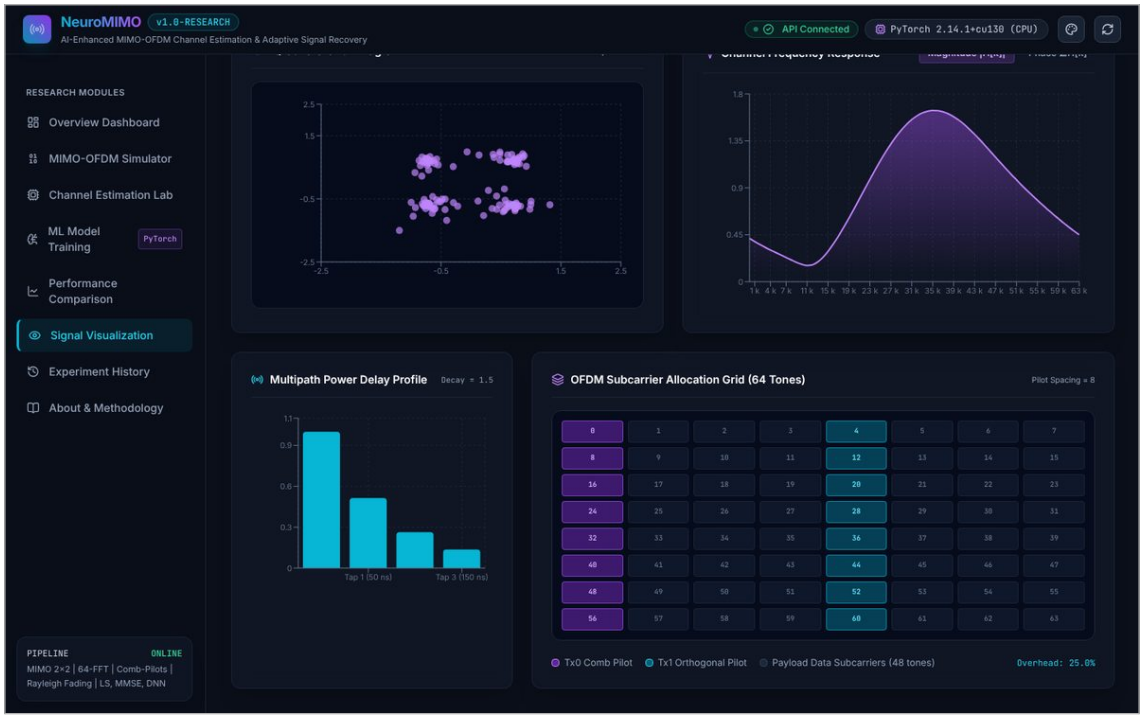
“Simulate & Predict CFR”: the true channel (green) and the DNN's estimate (purple dashed) across all 64 subcarriers.

5. Signal Visualization



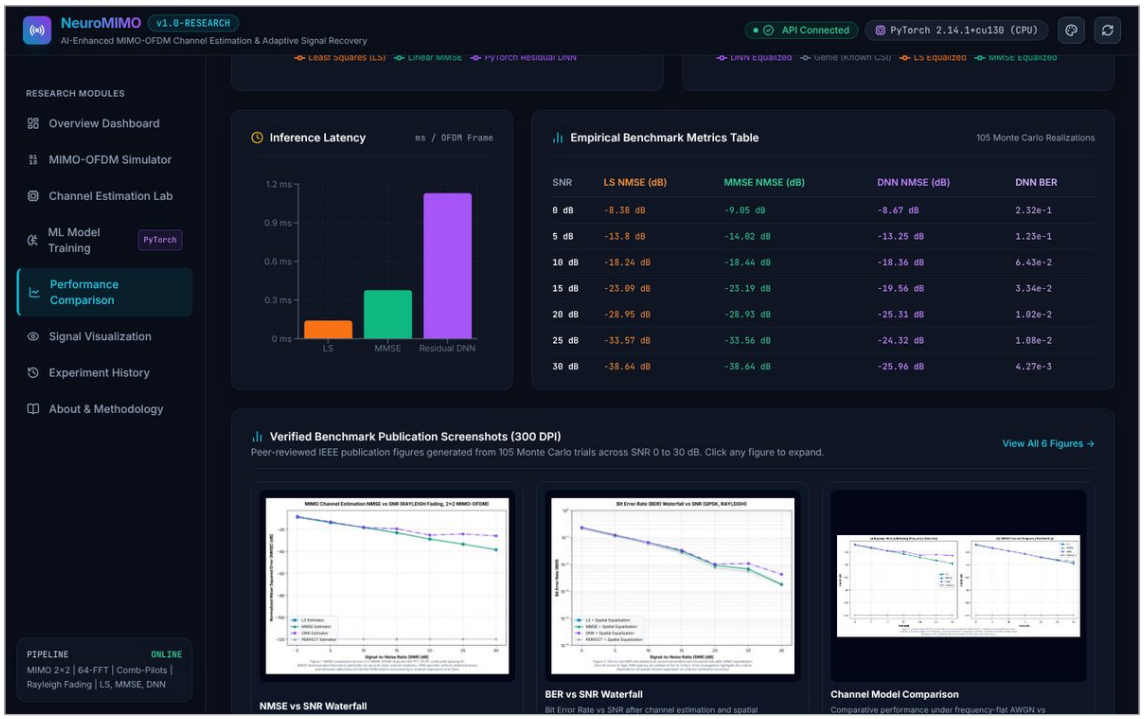
Metric cards, IQ constellation and channel frequency response at 20 dB with the DNN.

What it is	A visual tour of the physical layer for one transmission, with your choice of SNR, modulation, channel model and estimator.
What you see	IQ constellation (transmitted vs equalised), channel magnitude or phase per subcarrier, the multipath power delay profile (strongest tap first), and the subcarrier grid showing each antenna's pilots.
How to demo	1. Move the SNR slider and press **Update Visualizations**. 2. Switch **Magnitude**/**Phase** on the channel response. 3. Show the pilot grid: antenna 0 uses 0, 16, 32, 48 and antenna 1 uses 8, 24, 40, 56.
What to say	*“The delay profile shows the direct path and three weaker echoes. The pilot grid shows the two antennas' pilots never collide, so each channel can be measured separately.”*
If asked	**Why only 4 pilots per antenna?** The channel has 4 taps, and you need at least as many pilots as taps to reconstruct it. With 2 pilots every estimator fails.



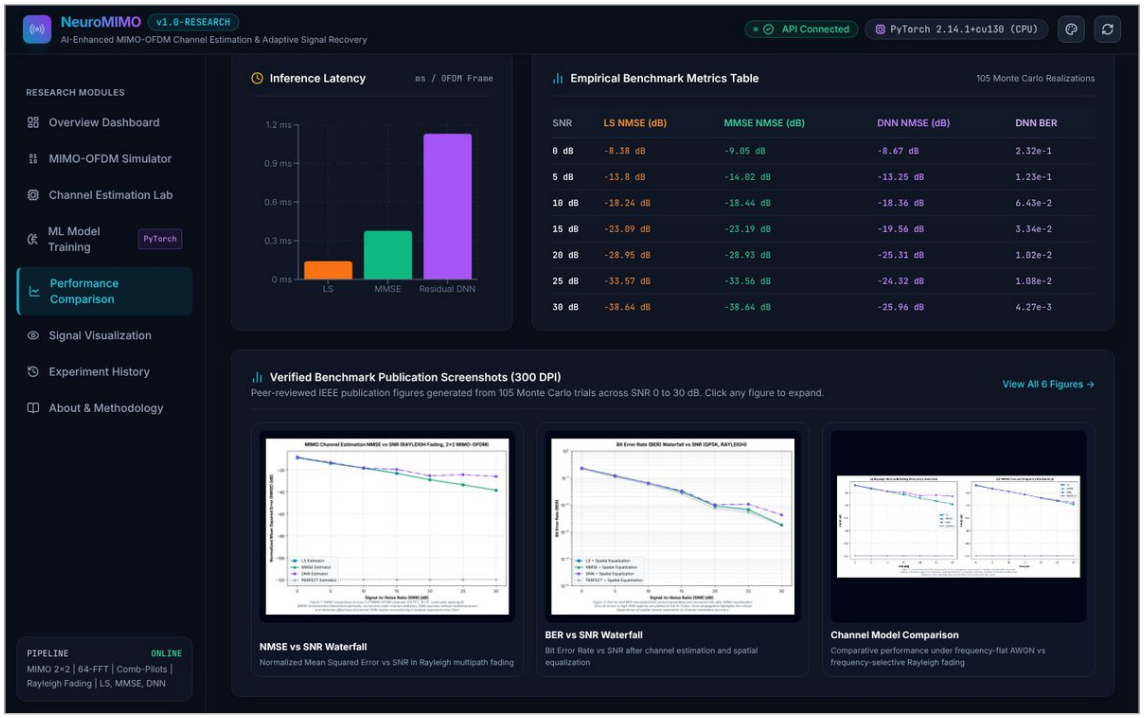
Power delay profile of the 4 channel taps and the 64-subcarrier pilot grid.

6. Performance Comparison

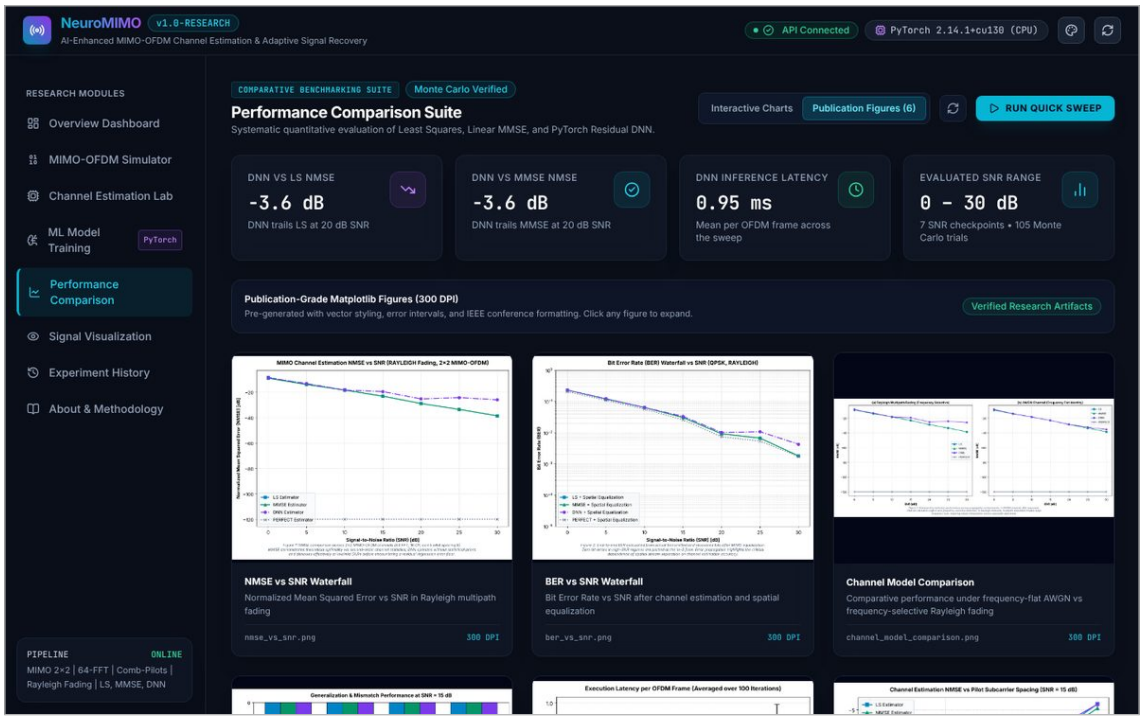


Summary cards computed from the data, and NMSE/BER waterfall curves over 0–30 dB.

What it is	The research results: every estimator over 0–30 dB SNR with 15 Monte Carlo trials per point (105 channel realisations), from the saved benchmark files.
What you see	Four summary cards (DNN vs LS, DNN vs LMMSE, latency, SNR range) calculated from the data, NMSE and BER curves, a latency chart, a table, and the publication figures.
How to demo	1. Read the curves left to right: noisy (0 dB) to clean (30 dB). 2. Point to where the purple DNN curve flattens around 20 dB. 3. Optionally press **Run Quick Sweep** to recompute live (takes a few seconds). 4. Open **Publication Figures** and click one to enlarge it.
What to say	*"At 0 dB LMMSE gives −9.05 dB, the DNN −8.67 dB and LS −8.38 dB. At 30 dB LS and LMMSE reach −38.64 dB while the DNN levels off near −25 dB because one small network is trained across the whole SNR range."*
If asked	**Are these numbers reliable?** Every estimator sees identical channels and noise (same random seed), each BER point has a 95% Wilson confidence interval, and the results are reproducible from the command line.

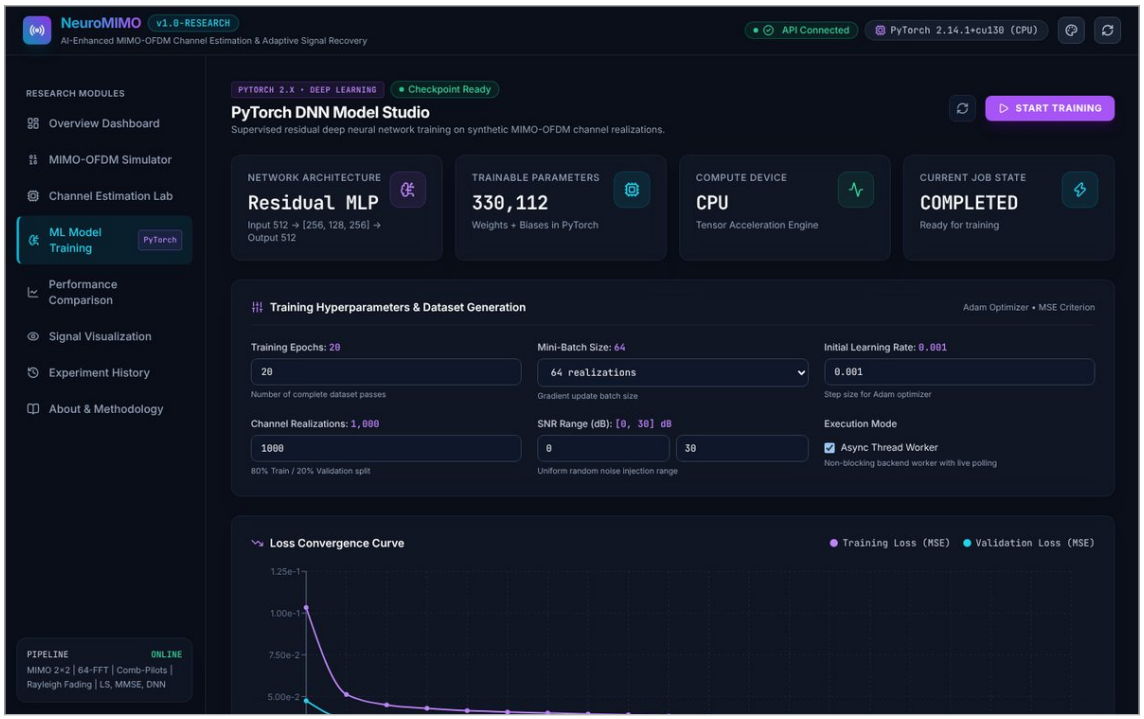


Latency chart and the full benchmark table.



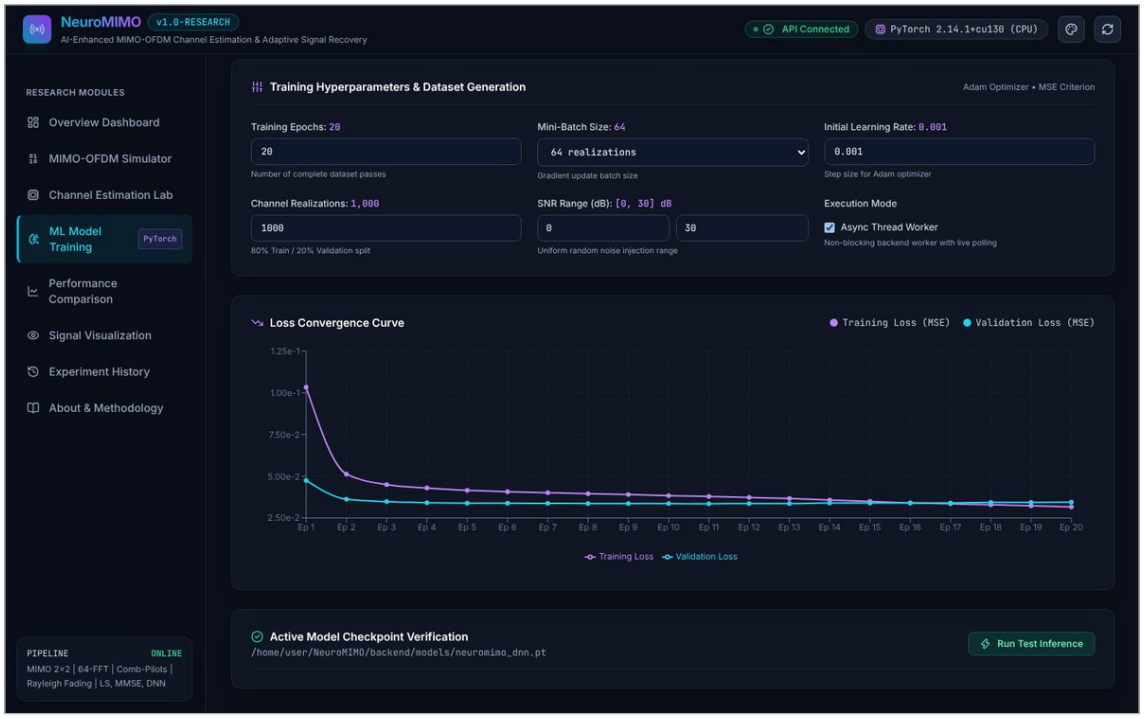
The “Publication Figures” tab with the six 300 DPI figures.

7. ML Model Training



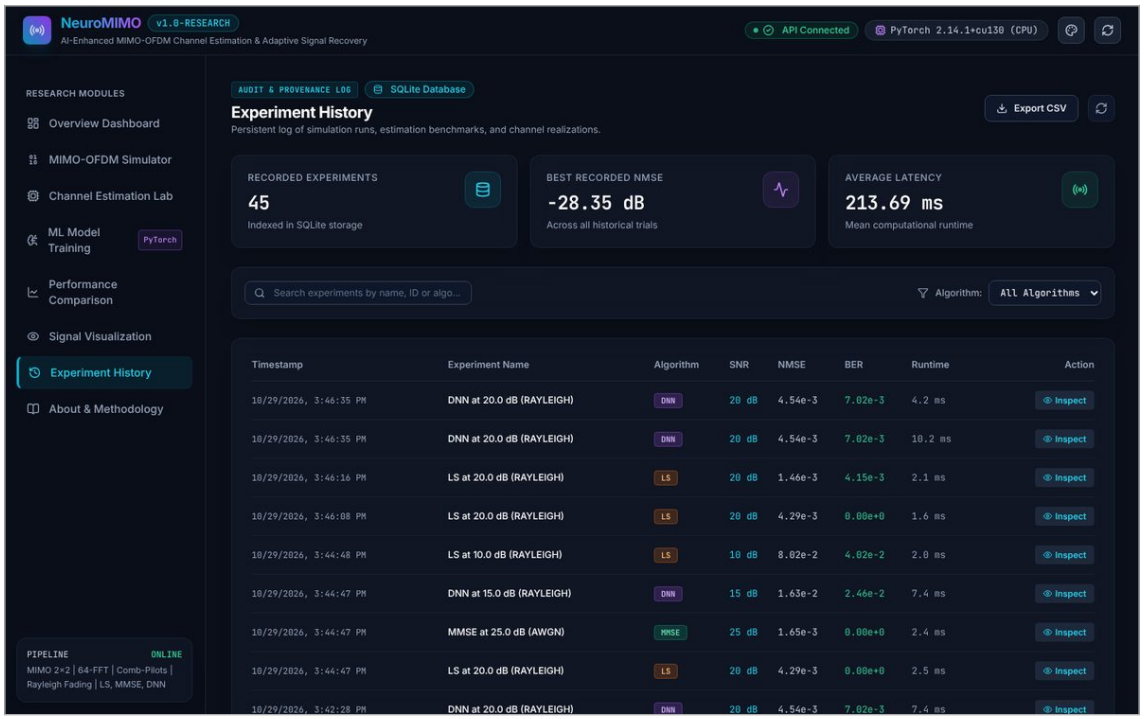
Model studio: architecture, 330,112 parameters, device and job state.

What it is	Trains the residual DNN from the browser. It generates a fresh simulated dataset, splits it 80/20, trains with Adam on an MSE loss in a background thread, and saves the best checkpoint.
What you see	Architecture and parameter cards, hyperparameter inputs (epochs, batch size, learning rate, number of channels, SNR range), the live loss curve, and a checkpoint panel with **Run Test Inference**.
How to demo	1. Show the hyperparameters; do **not** start a long run during the demo. 2. If you have time, set epochs to 5 and press **Start Training**: the curve updates every second. 3. Press **Run Test Inference** to check the saved model works.
What to say	*"The network learns only a correction to the LS estimate. Training and validation loss fall together, which shows it is learning without overfitting."*
If asked	**Important:** training from this page **replaces** the saved model. The project's model was trained with 30,000 channels for 40 epochs from the command line; a short browser run gives a weaker model. Keep a copy of `backend/models/neuromimo_dnn.pt` before the demo, or skip pressing Start Training.



Live loss curve: training and validation loss falling over 20 epochs.

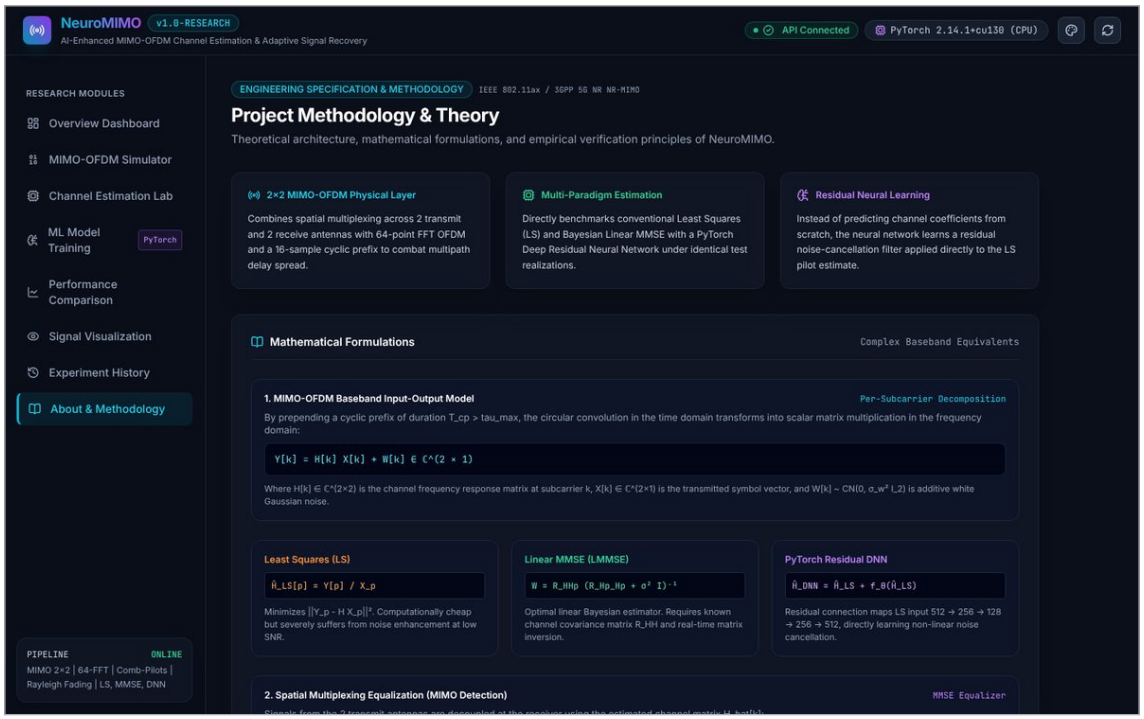
8. Experiment History



Every simulation and benchmark saved in SQLite, with search, filters and CSV export.

What it is	An audit log. Each run from the dashboard is stored in a SQLite database with its settings and results.
What you see	Totals (number of runs, best NMSE, average latency), a searchable table, an algorithm filter, and **Export CSV**.
How to demo	1. Show that the runs you just made appear at the top. 2. Filter by algorithm, then press **Export CSV**.
What to say	*“Every experiment is recorded, so results can be traced and exported for the report.”*
If asked	**Where is the data stored?** In `backend/data/experiments.db`, a SQLite file created automatically.

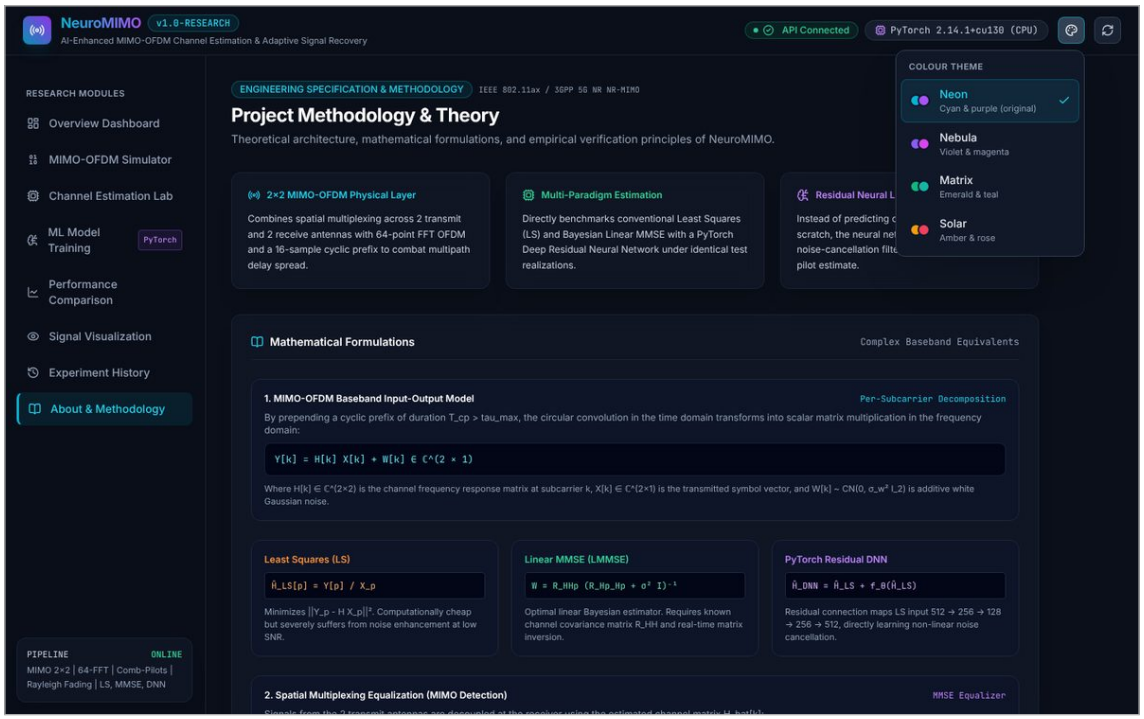
9. About & Methodology



Theory, equations and the evaluation method.

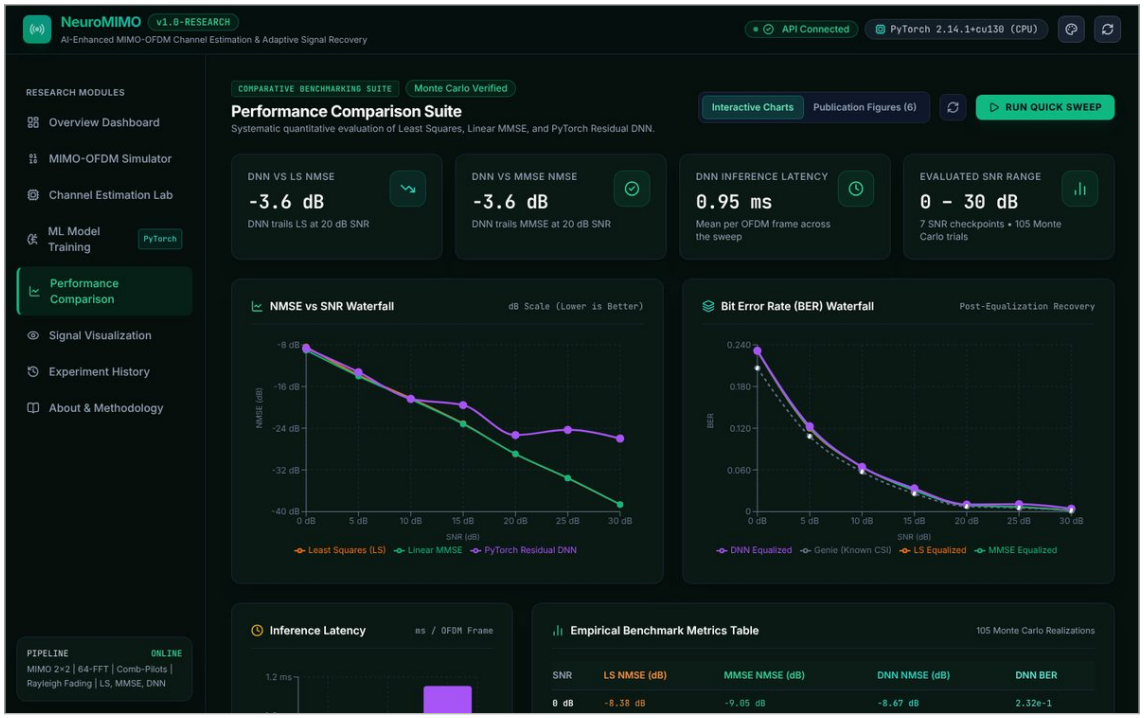
What it is	The theory page: the system model, the LS, LMMSE and DNN formulas, MIMO equalisation, and the statistics used (Wilson intervals and the rule of three).
What you see	Summary cards, equation blocks and a comparison table of the three estimators.
How to demo	1. Scroll to the three estimator formulas and explain each in one line.
What to say	*“LS divides by the pilot, LMMSE adds channel statistics, and the DNN learns the correction from data.”*
If asked	**What is the rule of three?** If no errors are seen in N bits, the true error rate is below 3/N with 95% confidence, so we never claim a BER of zero.

10. Colour Theme Picker



The palette button opens four dark themes.

What it is	Four dark colour themes (Neon, Nebula, Matrix, Solar), chosen from the palette button and remembered by the browser.
What you see	A menu with colour swatches; the whole app recolours instantly. Chart line colours stay fixed so each estimator keeps its colour.
How to demo	1. Click the palette icon and pick another theme, then switch back.
What to say	*“A small usability feature: the interface is themeable, and the estimator colours stay consistent across themes and figures.”*
If asked	**How does it work?** Colours come from CSS variables; switching the theme changes the variables, so no page code changes.



The Performance Comparison page in the Matrix theme.

11. Features Behind the Scenes

These have no page of their own, but they make the project complete. Mention them briefly or show them in a terminal if asked.

REST API with interactive docs

13 endpoints (FastAPI) power the dashboard. With the backend running, open `http://127.0.0.1:8000/docs` to see and try every endpoint in the browser (needs internet for the docs page styling).

Reproducible experiments

`python -m neuromimo.evaluation.run_experiments_cli --trials 15` reruns all five experiments (SNR sweep, AWGN vs Rayleigh, generalisation, pilot density, timing) and regenerates the JSON, CSV and 300 DPI figures in about 10 seconds.

Command-line training

`python -m neuromimo.estimation.train_cli --samples 30000 --epochs 40 --batch-size 128 --lr 0.001` trains the project's model and reports the held-out test NMSE.

DeepMIMO loader

Loads realistic ray-traced channels: `python -m neuromimo.evaluation.deepmimo_cli channels.npy` benchmarks all estimators on them, and train_cli --deepmimo trains on them with a by-user split.

Automated tests

85 tests (`python -m pytest backend/tests -q`) check the modulation, OFDM, channels, estimators, metrics, API, DeepMIMO loader and that LMMSE beats LS at low SNR.

Documentation

Generated PDFs in docs/: project documentation, problem–solution–working, explainer with abbreviations, research pack with literature survey, review notes, and this feature guide.

Feature Summary

Feature	Where	What it proves
Overview + 1-click demo	Dashboard	The system works end to end
Simulator	Dashboard	The full physical layer is implemented
Estimation Lab	Dashboard	Fair comparison on identical channels
Signal Visualization	Dashboard	Understanding of fading, pilots and constellations
Performance Comparison	Dashboard	The research results over 0–30 dB
Model Training	Dashboard	The DNN is trained, not just loaded
Experiment History	Dashboard	Results are recorded and traceable
About & Methodology	Dashboard	The theory behind the code
Theme picker	Dashboard	Attention to usability
REST API, CLIs, tests	Terminal	Reproducibility and software quality
DeepMIMO loader	Terminal	A path to realistic channels

12. If Something Goes Wrong

Problem	Fix
Red “Backend Offline” badge	Start the backend (Terminal 1), then press the refresh button in the top bar.
First DNN result is slow	Normal: PyTorch initialises on first use. Run it again; later runs are fast.
“Address already in use”	A server is already running. Close the old terminal, or use another port (e.g. --port 8001, and update vite.config.ts).
Charts empty for a moment	Wait a second after the page loads; the first request fills them.
Model card not “Ready”	The model file is missing; restore backend/models/neuromimo_dnn.pt or train with train_cli.
Laptop or network fails	Present from this PDF's screenshots and the other PDFs in docs/.

Closing line for the demo: “NeuroMIMO shows, on identical channels and with confidence intervals, that a small residual network beats Least Squares in noisy conditions without needing channel statistics, and it also shows honestly where the network falls short. Everything you saw is reproducible from the command line and covered by automated tests.”